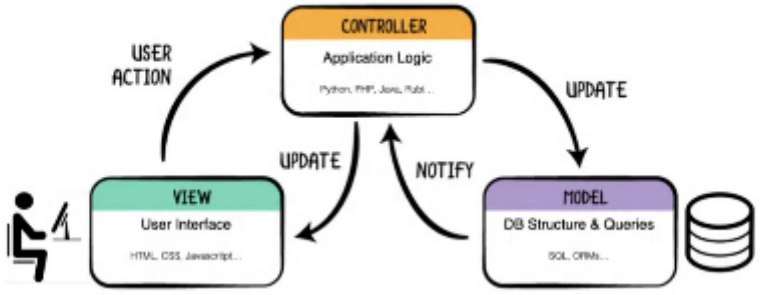
Introduzione a Flask

Introduzione

Flask è un micro web framework scritto in Python, sviluppato da Armin Ronacher e una parte del Pallets project.
Permette di svillupare applicazione web in maniera semplice.

Model View Controller (MVC)

Prima di affrontare Flask dobbiamo capire cos'è l'architettura MVC, per capire bene come Flask lavora.



MVC.png

L'**architettura MVC**

- **suddivide un'applicazione in tre componenti principali, con l'obiettivo di separare la logica di presentazione da quella di business e di gestione dei dati.**

1. View (Vista)

- **Ruolo: rappresenta l'interfaccia utente (UI).**
- **Tecnologie tipiche:** HTML, CSS, JavaScript.
- **Funzione: mostra i dati all'utente e riceve le sue azioni (input, click, form, ecc.), che vengono poi inviate al Controller.**

2. Controller

- **Ruolo: gestisce la logica applicativa e coordina il flusso di comunicazione tra View e Model.**
- **Tecnologie tipiche:** linguaggi lato server come Python (Flask, Django), Java, PHP, Ruby, ecc.

- **Funzione:** **riceve le richieste dell'utente dalla View, elabora i dati (eventualmente interagendo con il Model) e restituisce la risposta appropriata alla View.**

3. Model

- **Ruolo:** **rappresenta la struttura dei dati e la logica di accesso al database.**
- **Tecnologie tipiche:** SQL, ORM (Object-Relational Mapping), query al database.
- **Funzione:** **gestisce la memorizzazione, l'aggiornamento e il recupero delle informazioni.**

Flusso di interazione

1. **L'utente compie un'azione nella View (es. clicca un bottone).**
2. **La View invia la richiesta al Controller.**
3. **Il Controller elabora la richiesta e, se necessario, interagisce con il Model per leggere o modificare i dati.**
4. **Il Model aggiorna o fornisce i dati richiesti.**
5. **Il Controller riceve i risultati e aggiorna la View.**
6. **La View mostra i nuovi dati all'utente.**

Cos è Flask

Flask è un **micro-framework** per Python, chiamato così perché

- **non include nativamente un livello di gestione del database né funzionalità avanzate di validazione dei form, e richiede quindi poche librerie esterne.**
  Il suo compito è:
- **gestire la comunicazione tra il client (frontend) e il server (backend).**
  Quindi Flask offre al programmatore la **flessibilità di scegliere il database** con cui integrarsi e di utilizzare le librerie che preferisce per la validazione dei form.

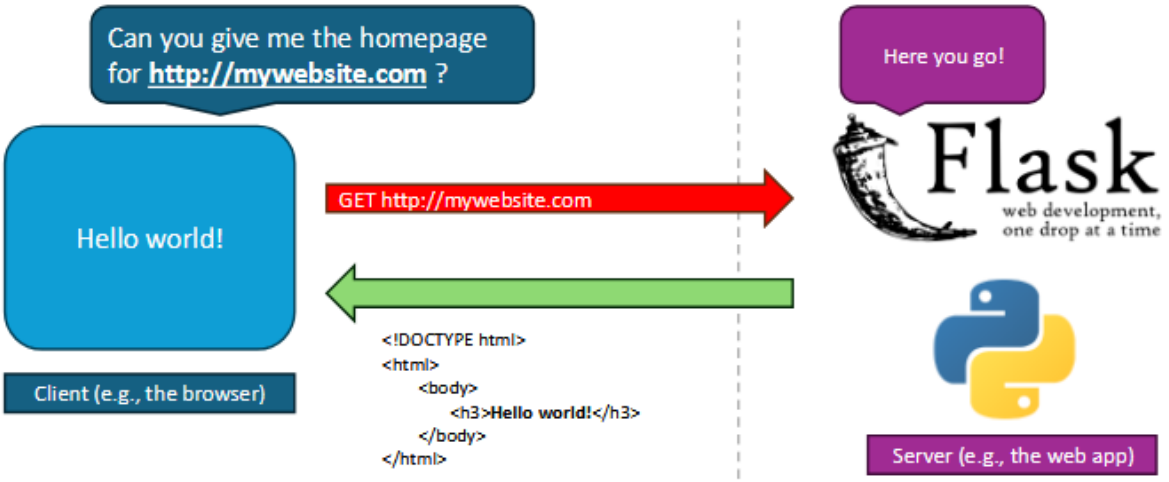
📋 Cos è la validazione dei form

La validazione dei form consiste nel controllare e verificare i dati inseriti dall'utente prima di inviarli al database, per garantire che siano

corretti e sicuri, evitando errori o l'iniezione di codice arbitrario.

Framework per costruire le web app in Python

Osservando lo schema:



Framework per costruire le web app in Python.png

Possiamo descrivere il processo passo per passo:

1. **Client (es. il browser)**
   - **L'utente digita un indirizzo web, ad esempio `http://mywebsite.com`, e invia una richiesta HTTP GET al server per ottenere la homepage.**
2. **Backend (Flask + Python)**
   - **L'app Flask riceve la richiesta, la elabora e individua quale funzione (detta *view function*) deve essere eseguita per rispondere alla route richiesta (`/` in questo caso).**

     Questa funzione restituisce una **risposta HTTP**, che può essere:
   - una pagina HTML generata dinamicamente, oppure
   - dati in formato JSON (nel caso di un'API), ecc.
3. **Server (Web Server o WSGI Server)**
   - **Il server riceve la risposta da Flask e la invia al client.**
   - In ambiente di sviluppo, Flask include un piccolo server integrato; in produzione, invece, si usa solitamente un server WSGI come *Gunicorn* o *uWSGI*.
4. **Client (Frontend)**
   - **Il browser riceve la risposta e la interpreta: se è HTML, renderizza la pagina visibile all'utente (es. "Hello, world!").**

Esempio Pratico:

Python

```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
  return "<h3>Hello, world!</h3>"

if __name__ == '__main__':
  app.run(debug=True)
```

Flusso di esecuzione:

1. Il browser invia `GET /` → `http://127.0.0.1:5000/`
2. Flask intercetta la richiesta e chiama `home()`
3. `home()` restituisce una risposta HTML
4. Flask invia la risposta al browser, che mostra “Hello, world!”

Installare flask

Per utilizzare Flask bisogna avere python installato maggiore alla versione 3.9
Di norma si installa tramite riga di comando (su windows e linux)

Shell

```shell
pip install flask
```

Warning

Se non funzionasse si puo eseguire con il comando (solo linux):

Shell

```shell
apt install flask
```

Scrivere la prima applicazione in Flask

Per usare flask su Python bisogna importare il framework `Flask` dal modulo `flask`.

Python

```python
from flask import Flask
```

Ecco il codice minimo per creare e avviare una semplice applicazione Flask:

Python

```python
from flask import Flask

# Creiamo un'istanza dell'app Flask
app = Flask(__name__)

# Avviamo il server di sviluppo
# debug=True attiva il debug mode (utile solo in fase di sviluppo)
# host='127.0.0.1' indica che il server sarà accessibile solo dal computer locale
# port=5000 specifica la porta su cui il server ascolta
app.run(debug=True, host='127.0.0.1', port=5000)
# In produzione, impostare debug=False

```

Spiegazione passo-passo

1. Importazione della classe Flask: `from flask import Flask`
   - **Questa riga importa la classe `Flask`, che serve per creare l'applicazione web.**
2. Creazione dell'istanza dell'applicazione: `app = Flask(__name__)`

- `__name__` **indica il nome del modulo corrente.**
- **Flask lo usa per capire dove cercare template, file statici e altre risorse.**
- In pratica, dice a Flask "questa è la mia applicazione principale".

1. Avvio del server di sviluppo: `app.run(debug=True, host='127.0.0.1', port=5000)`
   - `debug=True` abilita:

    - **il debug interattivo (per rilevare errori e vedere dettagli nel browser);**
    - **il reload automatico quando modifichi il codice;**
    - **la configurazione di sviluppo, con logging dettagliato.**
  - `host='127.0.0.1'` **limita l'accesso al solo computer locale.**
  - `port=5000` **specifica la porta su cui il server ascolta.**
    In produzione la modalità debug deve essere settata a False perché permette di eseguire codice arbitrario

Eseguire l'applicazione flask

Per eseguire il codice su terminale

Shell

```shell
python3 nomeFile.py
```

Il codice viene eseguito e viene creato l'oggetto flask con valore `__name__`

Definire le route statiche

In Flask, **ogni URL della tua web app corrisponde a una "route"**, cioè un percorso associato a una funzione Python che restituisce la risposta al client.

Per definire una route si usa il **decoratore** `@app.route`:

Python

```python
from flask import Flask
app = Flask(__name__)
@app.route('/')
def home() -> str:
    return "<h3>Hello, world!</h3>"
```

- La stringa `'/'`:
  - **indica l'URL della homepage.**
- La funzione `home`:
  - **definisce cosa viene restituito quando il client visita quell'URL.**
- Per default, `@app.route` **risponde a richieste HTTP GET, quindi è un metodo statico.**

Avvio dell'applicazione

Ci sono due modi principali per avviare un'app Flask:

1. **Usando `app.run()` nel file Python**

- Se il tuo file si chiama `main.py` e contiene

Python

```python
if __name__ == "__main__":
    app.run()
```

- Puoi avviare l'app con:

Shell

```shell
python main.py
```

- **Questo metodo funziona, ma ogni volta che cambi valori di default nel codice (es. porta o host) devi salvare e riavviare il file.**

2. **senza `app.run()`, usando il comando `flask`**
   Se elimini `app.run()`, Flask può essere eseguito direttamente dalla CLI:

Shell

```shell
flask --app main run --port 4000
```

- `--app main` **indica il file Python contenente l'app (main.py).**
- `--port 4000` **permette di cambiare la porta senza modificare il codice.**

Un'alternativa è rinominare il file in **app.py**. In questo caso:

Shell

```shell
flask run --port 4000
```

- **Non serve specificare il nome del file, Flask lo rileva automaticamente.**
- **Possiamo comunque cambiare porta, host o altre opzioni da CLI.**

Sintesi

- Una **route statica** collega una funzione a un URL fisso.
- Ogni funzione associata a una route gestisce la risposta per quel percorso.
- Usare il comando `flask run` senza `app.run()` è più flessibile, soprattutto durante lo sviluppo, perché permette di modificare porta, host e altre impostazioni senza cambiare il codice.

Route dinamiche in Flask

Nel mondo reale, la maggior parte degli URL non è statica: ad esempio, vogliamo mostrare informazioni **personalizzate per ciascun utente** o **recuperare dati specifici** in base a un identificatore.
In Flask, questo si realizza tramite **route dinamiche:**

- **usando sempre il decoratore `@app.route` con variabili nel path.**
  Esempio di route dinamiche

Python

```python
@app.route('/user/<string:username>')
def show_user_profile(username: str) -> str:
    return f"Profilo di {username}"

@app.route('/post/<int:post_id>')
def show_post(post_id: int) -> str:
    return f"Post {post_id}"
```

- La parte `<string:username>`:
  - **indica che tutto ciò che compare in quella posizione dell'URL verrà passato come argomento alla funzione `show_user_profile`.**
- Analogamente `<int:post_id>`:
  - **converte automaticamente la variabile in intero e la passa alla funzione `show_post`.**

Esempi di URL e Output

URL	Funzione chiamata	Risultato
`/user/Marco`	`show_user_profile("Marco")`	`"Profilo di Marco"`

URL	Funzione chiamata	Risultato
`/post/25`	`show_post(25)`	`"Post 25"`

Tipi supportati per le variabili dinamiche

Se non specifichiamo un tipo, Flask assume **stringa** (`string`).
Altri tipi possibili:

- `int` → **numeri interi**
- `float` → **numeri con la virgola**
- `path` → **permette anche gli slash** `/` **all'interno della variabile**
- `uuid` → **identificatori universali unici**

Piu variabili in una route

È possibile avere **più variabili nello stesso URL**.
Ad esempio:

Python

```python
@app.route('/user/<string:username>/age/<int:age>')
def show_user_age(username: str, age: int) -> str:
    return f"Profilo di {username}: {age} anni"
```

- URL: `/user/Marco/age/18`
- Risultato: `"Profilo di Marco: 18 anni"`

Le variabili dinamiche **devono sempre corrispondere agli argomenti della funzione**. Se manca un argomento o c'è un tipo incompatibile, Flask genera un errore.

Decoratori specifici per il metodo HTTP

Per default, `@app.route` risponde a **GET**.
Se vogliamo specificare esplicitamente il metodo, possiamo usare:

Python

```python
@app.get('/user/<string:username>')   # solo GET
@app.post('/user/<string:username>')  # solo POST
```

- Questo rende il codice più chiaro e **assicura che la route risponda solo al metodo desiderato**.
- È particolarmente utile quando abbiamo più route sullo stesso URL ma con comportamenti diversi (GET per leggere, POST per creare, PUT per aggiornare, DELETE per eliminare).

`url_for()` — Generare dinamicamente gli URL

Quando si sviluppano applicazioni web, spesso gli URL **non sono statici** e contengono variabili come ID o nomi utenti.
Scrivere manualmente tutti i percorsi può diventare complesso e soggetto a errori, specialmente se cambiano le route nel tempo.

Flask offre la funzione `url_for()`, che permette di:

- **ricostruire automaticamente l'URL associato a una funzione di view, evitando di dover scrivere i percorsi manualmente.**

✓ Vantaggi Principali

1. **Astrazione dal percorso fisico:**
   - Non serve ricordare o riscrivere manualmente la route.
   - Flask genera l'URL usando il **nome della funzione**.
2. **Manutenzione semplificata:**
   - se una route cambia nel codice, `url_for()` aggiorna automaticamente tutti i riferimenti.
3. **Percorsi sempre assoluti:**
   - i link generati sono sempre completi (non relativi), riducendo gli errori di navigazione.
4. **Gestione automatica dei caratteri speciali:**
   - gli spazi o simboli vengono convertiti in codifica URL-safe (es. `John Doe` → `John%20Doe`).

Esempio Pratico:

Python

```python
from flask import Flask, url_for
app = Flask(__name__)
```

```python

# Definisce la route principale (`home`) necessaria come punto di ingresso dell'applicazione.
@app.route('/')
def home():
    return "Homepage"

#Cattura un parametro di tipo stringa, correttamente tipizzato, e lo mostra nella risposta HTML.
@app.route('/user/<string:username>')
def show_user_profile(username):
    return f"Profilo utente: {username}"

#Cattura un parametro numerico (`int`) e lo restituisce nella risposta.
@app.route('/post/<int:post_id>')
def show_post(post_id):
    return f"Post n° {post_id}"

# Utilizzato nel contesto di test (`app.test_request_context()`), è il modo corretto per costruire dinamicamente gli URL delle route definite.

with app.test_request_context():
    print(url_for('home'))
    print(url_for('show_user_profile', username='John Doe'))
    print(url_for('show_post', post_id=42))

```

Expected Output:

Shell

```
/
/user/John%20Doe
/post/42
```

Come funziona

- `url_for('home')` → **genera** `/`**, la route della homepage.**
- `url_for('show_user_profile', username='John Doe')` → **sostituisce** `<username>` **con** `"John Doe"`**, producendo** `/user/John%20Doe`**.**

- `url_for('show_post', post_id=42)` → **sostituisce** `<post_id>` **con** `42`**, generando** `/post/42`**.**

In pratica, `url_for()` :

- **mappa il nome della funzione sulla route corrispondente, sostituendo automaticamente eventuali parametri dinamici e producendo URL validi e sicuri, senza bisogno di scrivere manualmente il percorso.**